

REPORTE DE PRÁCTICA: CREACIÓN Y GESTIÓN DE DEMONIOS PERSISTENTES CON SYSTEMD EN GNU/LINUX

Asignatura: Sistemas Operativos

Tema: Gestión de Servicios, Supervisor Systemd, Control de Procesos y Señales IPC

1. RESUMEN Y OBJETIVOS

El presente laboratorio analiza la implementación de un proceso en segundo plano (demonio) administrado por el sistema de inicialización y supervisor de servicios de GNU/Linux, systemd. El objetivo central es crear un script de monitoreo continuo de métricas del sistema, aislarlo como un servicio autónomo mediante un archivo de unidad (.service), y validar las políticas de resiliencia y alta disponibilidad del supervisor frente a terminaciones abruptas (señales **SIGKILL**), justificando teóricamente el control de procesos, los grupos de control (**cgroups**) y el restablecimiento automático de PIDs.

2. REGISTRO EXPERIMENTAL SINTÉTICO

Etapas	Comandos Clave	Evento / Salida Observable
1. Creación e Inicialización	<code>sudo nano /usr/local/bin/monitor-salud.sh</code>	Se escribe el script con un ciclo while true y pausas de 5 segundos.
	<code>sudo chmod +x -v /usr/local/bin/monitor-salud.sh</code>	Modo modificado a 0755 (rwxr-xr-x), garantizando ejecución.
	<code>/usr/local/bin/monitor-salud.sh</code>	Prueba manual exitosa. Salida formateada con Fecha, Procesos y RAM.
2. Registro del Demonio en Systemd	<code>sudo nano /etc/systemd/system/monitor-salud.service</code>	Creación de la unidad de servicio con política Restart=always.
	<code>sudo systemctl daemon-reload</code>	El supervisor recarga su configuración en memoria.
	<code>sudo systemctl start monitor-salud.service</code>	El demonio inicia; systemctl status reporta estado

		"active (running)".
	sudo systemctl enable monitor-salud.service	Creación del enlace simbólico para persistencia tras el arranque.
3. Prueba de Resiliencia y Control de Fallos	sudo kill -9 7868	Envío de señal de terminación forzada al PID principal.
	systemctl status monitor-salud.service	El servicio pasa brevemente a "activating (auto-restart) (Result: signal)" tras recibir signal=KILL.
	systemctl status monitor-salud.service	El servicio levanta automáticamente en menos de 1 segundo con un nuevo PID (7997).

3. ANÁLISIS REFLEXIVO Y JUSTIFICACIÓN TEÓRICA

3.1. Aislamiento del Proceso y Supervisión (Systemd y Cgroups)

Un demonio tradicional en UNIX debía realizar una técnica conocida como "doble fork" para desvincularse de la terminal (TTY) y ser adoptado por el proceso inicial. En la arquitectura moderna, systemd simplifica esto. Al definir Type=simple, systemd ejecuta directamente el binario /usr/local/bin/monitor-salud.sh y encapsula el proceso resultante dentro de un grupo de control (Control Group o **cgroup**), específicamente en /system.slice/monitor-salud.service.

Justificación Teórica: El uso de **cgroups** asegura que, sin importar cuántos subprocesos genere el script, systemd mantiene un rastreo absoluto de su jerarquía y consumo de recursos, impidiendo la creación de procesos huérfanos o "zombies".

3.2. Extracción de Métricas del Kernel en Tiempo de Ejecución

El script diseñado no calcula las métricas por sí mismo, sino que consulta interfaces expuestas por el núcleo.

- Procesos activos: ps -e lee iterativamente la tabla de procesos del directorio de pseudo-archivos /proc.
- RAM disponible: free -h parsea el archivo /proc/meminfo.

Justificación: El script actúa como una envoltura de espacio de usuario (user-space wrapper) que utiliza tuberías (**pipes**) para redirigir el flujo de salida estándar (**stdout**) de estas herramientas a utilidades de procesamiento de texto (wc, awk), empaquetándolas eficientemente sin incurrir en un alto consumo de CPU debido a la directiva sleep 5.

3.3. Políticas de Resiliencia frente a Interrupciones Asíncronas (SIGKILL)

Observación: El profesor o administrador envía kill -9 7868. En lugar de que el servicio muera permanentemente, revive al instante con el **PID 7997** .

Justificación a nivel de Kernel y Supervisor:

1. Inmunidad a Intercepciones: La opción -9 corresponde a la señal asíncrona **SIGKILL** . A diferencia de **SIGTERM** (**-15**), **SIGKILL** no puede ser bloqueada, ignorada o manejada por el script. El kernel asume la responsabilidad de destruir el contexto del proceso y liberar su memoria inmediatamente.
2. Detección de Estado (SIGCHLD): Cuando el proceso **7868** muere, el kernel notifica a su padre (systemd, que suele tener **PID 1**) mediante una señal **SIGCHLD** .
3. Disparo de la Directiva Restart: Al recibir el cambio de estado, systemd lee el código de salida (marcado como code=killed, signal=KILL). Como el archivo .service dictamina Restart=always, el supervisor hace caso omiso del motivo de la muerte y programa la reejecución basándose en RestartSec=1.
4. Asignación de **PID** : El sistema operativo asigna el siguiente identificador de proceso secuencial libre (**7997**), demostrando que se trata de una instancia completamente nueva y no de una reanudación del proceso anterior.
5. CONCLUSIONES
6. Centralización de Servicios: La utilización de systemd estandariza el ciclo de vida del software en segundo plano, reemplazando complejos scripts de inicialización SysV por archivos de configuración declarativos (.service).
7. Robustez del Sistema Operativo: Se demuestra que las directivas de auto-recuperación (Restart=always) son fundamentales para la alta disponibilidad, garantizando que el monitoreo crítico continúe operando incluso ante errores fatales o intervenciones manuales destructivas (**SIGKILL**).
8. Jerarquía y Rastreo: Gracias al modelo de Control Groups de Linux, las herramientas modernas de supervisión pueden gobernar los procesos con un aislamiento estricto, previniendo fugas de recursos y facilitando la administración de sistemas a nivel empresarial.